

SIMATS ENGINEERING

CO3 - ASSESSMENT TOOL 1

Case Study

COURSE NAME: DEEP LEARNING

COURSE CODE: MLA0408

COURSE OUTCOME COVERED

CO3: Analyze and apply convolutional neural network architectures, including convolutional layers, filters, parameter sharing, regularization, AlexNet and ResNet, to develop solutions for image classification and real-world computer vision applications. (L4)

CO Weightage: 25%

Assessment Tool Weightage: 25%

QUESTIONS:

Total Marks: 25

Case Study 1: CNN for Automated Medical Image Classification

A healthcare organization wants to develop a deep learning system to automatically classify chest X-ray images into **normal** and **disease-affected** categories. The dataset contains thousands of X-ray images with variations in size, brightness and image quality. A CNN model is proposed because it can automatically learn important visual features such as edges, textures, and abnormal regions without manually designing features.

Question:

1. Design and explain a suitable **CNN architecture** for this medical image classification system.
2. Describe the **motivation for using CNNs** and explain the role of **convolutional layers, filters, pooling layers, and fully connected layers** in the proposed architecture.
3. Explain how **parameter sharing** reduces the number of trainable parameters compared with a fully connected network.
4. Discuss suitable **regularization techniques such as dropout, data augmentation and batch normalization** to reduce overfitting.
5. Finally, explain whether **AlexNet or ResNet** would be more appropriate for this application and justify your choice based on feature learning, network depth, computational requirements and classification performance.

Case Study 2: CNN-Based Autonomous Vehicle Object Recognition

An autonomous vehicle company is developing a vision system that must identify **cars, pedestrians, traffic signs, bicycles, and road obstacles** from images captured by cameras mounted on the vehicle. The system must recognize objects under different lighting conditions, viewing angles, distances and road environments. The development team is considering **AlexNet and ResNet** architectures for building the CNN-based recognition system.

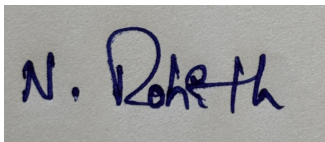
Question:

1. Analyze how a **CNN architecture** can be designed for this autonomous vehicle application.
2. Explain the purpose of the **input, convolution, activation, pooling, and fully connected/output layers** and describe how convolutional **filters progressively learn low-level and high-level features**.
3. Explain the importance of **parameter sharing** in reducing computational complexity and improving translation-related feature detection.
4. Discuss the possible problem of **overfitting** and recommend suitable regularization methods.
5. Compare **AlexNet and ResNet** in terms of architecture, depth, parameter efficiency, training difficulty, vanishing-gradient problems and feature-learning capability.
6. Finally, recommend the most suitable architecture for the autonomous vehicle system and justify your decision based on **accuracy, computational cost and real-time application requirements**.

Code of Conduct:

*I NALIPI REDDY ROHITH/192425115(Name / Reg No) certify that this submission is my original work and that I have adhered to the guidelines specified for this assessment.
I understand that any violation of academic integrity rules will result in disciplinary action.*

Signature of the student:

A rectangular box containing a handwritten signature in blue ink. The signature appears to be 'N. Rohith'.

RUBRICS FOR CALCULATION:

Criteria	Weightage	Level 4 – Exemplary	Level 3 – Proficient	Level 2 – Developing	Level 1 – Beginning
Understanding of Case	25	Thorough understanding; clearly identifies all key issues	Good understanding with most issues identified	Basic understanding; some issues missed	Poor understanding; problem not identified correctly
Application of Concepts	25	Effectively applies relevant concepts to analyze the case deeply	Applies appropriate concepts with minor gaps	Limited application of concepts	Fails to apply relevant concepts
Analysis	20	In-depth analysis with strong reasoning and insights	Good analysis with logical reasoning	Basic analysis with limited depth	Weak or no analysis; lacks reasoning
Solution Design	20	Innovative, feasible solutions with strong justification	Logical solutions with adequate justification	Basic solutions with weak justification	Incorrect or no solution provided
Clarity	10	Clear, well-structured, and easy to understand	Organized and understandable presentation	Limited clarity and structure	Poor clarity; difficult to understand

Case Study 1: CNN for Automated Medical Image Classification

1. Suitable CNN Architecture

A suitable CNN architecture for chest X-ray classification can be:

Input Image → Convolution → ReLU → Pooling → Convolution → ReLU → Pooling → Convolution → ReLU → Pooling → Fully Connected → Dropout → Output

Example:

- **Input:** $224 \times 224 \times 1$ grayscale X-ray image
- **Conv Layer 1:** 32 filters, 3×3
- **ReLU activation**
- **Max Pooling:** 2×2
- **Conv Layer 2:** 64 filters, 3×3
- **ReLU activation**
- **Max Pooling:** 2×2
- **Conv Layer 3:** 128 filters, 3×3
- **ReLU activation**
- **Max Pooling:** 2×2
- **Fully Connected Layer:** 128 neurons
- **Dropout:** 0.5
- **Output Layer:** 2 neurons using Softmax

The network progressively learns simple features such as edges in early layers and complex features such as textures and abnormal lung regions in deeper layers.

2. Motivation and Role of CNN Components

Motivation for CNN

CNNs are suitable because they can:

- Automatically learn important image features.
- Handle spatial relationships between pixels.
- Detect edges, textures and shapes.
- Avoid manual feature extraction.
- Provide high classification accuracy for medical images.

Convolutional Layers

Convolutional layers apply filters to the image to detect important features such as:

- Edges
- Lines
- Textures
- Abnormal patterns

Filters

Filters are small matrices that slide over the image and produce **feature maps**. Different filters learn different visual patterns.

Pooling Layers

Pooling reduces the spatial size of feature maps.

Max pooling keeps the maximum value from a region.

Benefits:

- Reduces computation.
- Reduces memory usage.
- Helps prevent overfitting.
- Makes feature detection less sensitive to small position changes.

Fully Connected Layers

The extracted features are converted into a vector and passed to fully connected layers. These layers perform the final classification.

For this application:

Normal / Disease-Affected

3. Parameter Sharing

In a fully connected network, every neuron may connect to every pixel.

For example, a 224×224 image has:

$224 \times 224 = 50,176$ pixels

Connecting this to 100 neurons requires approximately:

$50,176 \times 100 = 5,017,600$ weights

In CNNs, the same filter is reused at different positions of the image. Therefore, the filter does not require separate weights for every location.

For example, a **3×3 filter** has only:

$3 \times 3 = 9$ weights + 1 bias

This is called **parameter sharing**.

Therefore, CNNs require significantly fewer trainable parameters and computational resources than fully connected networks.

4. Regularization Techniques

Dropout

Dropout randomly disables some neurons during training.

Example:

Dropout = 0.5

This prevents the network from depending too much on particular neurons and reduces overfitting.

Data Augmentation

Medical images can be augmented using suitable transformations such as:

- Small rotations
- Scaling
- Translation
- Brightness adjustment
- Contrast adjustment

This creates additional training variations and improves generalization.

Care must be taken that augmentation does not create medically unrealistic X-rays.

Batch Normalization

Batch normalization normalizes intermediate activations during training.

Benefits:

- Stabilizes training.
- Allows faster convergence.

- Reduces sensitivity to initialization.
- Can provide some regularization.

5. AlexNet vs ResNet

Feature	AlexNet	ResNet
Depth	Relatively shallow	Much deeper
Feature learning	Good	Excellent
Training	Easier	Easier for deep networks because of residual connections
Vanishing gradient	More possible	Strongly reduced
Accuracy	Good	Generally higher
Parameters	More for its capacity	Modern variants can be parameter-efficient
Computational cost	Lower	Higher depending on version
Medical image classification	Suitable	More suitable

Recommended: ResNet

ResNet is more appropriate for this application because medical X-rays can contain subtle and complex patterns. Its deeper architecture can learn more sophisticated features.

Residual/skip connections allow information and gradients to flow through very deep networks, reducing the **vanishing-gradient problem**.

Therefore:

ResNet → **Better feature learning** → **Better classification performance**

A smaller ResNet such as **ResNet-18** or **ResNet-34** can be selected when computational resources are limited.

Case Study 2: CNN-Based Autonomous Vehicle Object Recognition

1. CNN Architecture

A suitable architecture for autonomous vehicle object recognition is:

Camera Image → Convolution → ReLU → Pooling → Convolution → ReLU → Pooling → Deep Convolution Layers → Fully Connected/Detection Head → Output

Input images can contain multiple objects such as:

- Cars
- Pedestrians
- Bicycles
- Traffic signs
- Obstacles

For a simple classification system:

Input → CNN Feature Extractor → Fully Connected Layer → Softmax Output

For a practical autonomous vehicle system, an object-detection architecture would be more appropriate because it must identify **where** each object is located as well as its class.

2. CNN Layer Functions

Input Layer

Receives images captured by vehicle cameras.

Example:

224 × 224 × 3 RGB image

Convolution Layer

Applies filters to detect visual patterns.

Early layers detect:

- Edges
- Lines
- Corners

Middle layers detect:

- Shapes
- Textures
- Vehicle parts
- Pedestrian structures

Deep layers detect:

- Cars
- Pedestrians
- Traffic signs
- Complex road objects

Activation Layer

Usually uses **ReLU**:

$$\text{ReLU}(x) = \max(0, x)$$

It introduces non-linearity and allows the network to learn complex relationships.

Pooling Layer

Reduces feature-map dimensions while retaining important information.

It reduces:

- Computation
- Memory requirements

Fully Connected / Output Layer

Uses extracted features to classify the detected object.

Example output:

Car | Pedestrian | Bicycle | Traffic Sign | Obstacle

3. Parameter Sharing

CNN filters are reused across different locations of an image.

For example, a single edge-detection filter can detect an edge on the:

- Left side
- Center

- Right side

of the image.

Therefore, the same parameters are used throughout the image.

Benefits:

- Fewer trainable parameters.
- Lower memory requirement.
- Lower computational complexity.
- Faster training.
- Ability to detect the same feature at different positions.

This is particularly important for autonomous vehicles because objects can appear at different positions in camera images.

4. Overfitting and Regularization

Autonomous vehicle datasets can contain thousands of images, but the network may still memorize training examples.

Suitable techniques include:

Data Augmentation

Use:

- Rotation
- Scaling
- Cropping
- Horizontal flipping where appropriate
- Brightness changes
- Contrast changes
- Different weather/lighting simulations

This helps the model handle different road conditions.

Dropout

Randomly disables neurons during training and reduces dependence on specific features.

Batch Normalization

Normalizes intermediate activations and helps stabilize and speed up training.

Early Stopping

Training can be stopped when validation performance stops improving.

Weight Decay

Adds a penalty for excessively large weights and helps control model complexity.

5. AlexNet vs ResNet

Feature	AlexNet	ResNet
Architecture	Traditional CNN	CNN with residual/skip connections
Depth	8 learnable layers	Can have 18, 34, 50, 101+ layers
Feature learning	Good	Excellent
Vanishing gradient	More likely	Significantly reduced
Training deep networks	Difficult	Easier
Accuracy	Good	Generally higher
Computational cost	Lower	Depends on ResNet version
Complex objects	Limited compared with modern networks	Better
Autonomous driving	Basic applications	Better suited

Vanishing Gradient Problem

In very deep networks, gradients can become extremely small while propagating backward.

This makes early layers difficult to train.

ResNet solves this using **skip connections**:

Input → Several layers → Output

along with a direct shortcut:

Input —————→ Output

This allows gradients to flow more effectively through the network.

6. Recommended Architecture

Recommended: ResNet

For autonomous vehicle object recognition, **ResNet is generally the better choice** because:

1. **Higher accuracy** – It can learn complex visual features.
2. **Better feature extraction** – Deep layers can identify complex objects.
3. **Reduced vanishing-gradient problem** – Residual connections improve training.
4. **Better generalization** – Useful for different lighting, distances and environments.
5. **Transfer learning** – Pre-trained ResNet models can be fine-tuned on road-image datasets.

However, real-time processing is extremely important in autonomous vehicles.

Therefore, instead of selecting a very large ResNet, a **smaller ResNet such as ResNet-18/34** can provide a good balance between:

Accuracy + Computational Cost + Real-Time Performance

Final Choice

ResNet-18/34 → suitable for resource-constrained real-time systems

Larger ResNet → better when high computational power and maximum accuracy are available.

For an actual autonomous vehicle, a dedicated real-time **object detection model** would normally be preferred over plain image classification because the system must determine both **what the object is and where it is**.